Approvo.

Riguardo all'alternativa 'solo Compose' : possiamo effettivamente valutarla in quanto la versione legacy non e' per nulla vincolante da qui in avanti, naturalmente dall'esecuzione di un singolo servizio come comando one-off con Compose devo ottenere lo stesso container generato con 40.run.sh

Sì: possiamo adottare **Compose come unico backend**, anche quando viene eseguito soltanto il container di sviluppo. `docker compose run` crea un nuovo container utilizzando la configurazione del servizio ed è adatto alle sessioni one-off; con `--rm` lo rimuove all'uscita.[1]

Il requisito da fissare è ottenere un container equivalente a quello avviato dall'attuale script di run, non un container identico in ogni dettaglio Docker: Compose aggiunge una propria gestione dello stack e può introdurre differenze di rete e metadati.

1. Che cosa deve rimanere equivalente

La migrazione deve preservare questi aspetti:

Aspetto	Comportamento richiesto
Immagine	La stessa `IMAGE_NAME` risolta da `docker-dev`
Nome container	Lo stesso `CONTAINER_NAME`, se vogliamo preservare l'identità attuale
Hostname	Lo stesso hostname configurato oggi
Utente	Quello definito nell'immagine, salvo override esplicito
Working directory	`/app`
Entrypoint	Quello dell'immagine Python
Mount	Progetto, virtualenv, `uv-python` e aggiunte del progetto
Ambiente e risorse	Valori risultanti dai due livelli `runtime.yaml`
Comando	Gli stessi argomenti, conservati come argv
Interattività	Sessione interattiva quando richiesta
Rimozione	Container eliminato all'uscita
Porte	Soltanto quelle dichiarate esplicitamente

Non occorre sovrascrivere `USER` o `ENTRYPOINT` nel Compose generato: tali default appartengono già all'immagine. Docker usa l'utente dell'immagine per eseguire il processo a runtime e consente di mantenere l'entrypoint passando soltanto gli argomenti del comando.[2]

2. Il servizio `dev` è una definizione, non un container residente

Il file Compose descrive il servizio:

```
services:
  dev:
    image: dev.python
    hostname: dev.python.foo
    stdin_open: true
    tty: true
    working_dir: /app

    volumes:
      - type: bind
        source: /percorso/assoluto/progetto
        target: /app

      - type: volume
        source: python-venv
        target: /app/.venv

      - type: volume
        source: uv-python
        target: /home/utente/.local/share/uv/python

volumes:
  python-venv:
    external: true
    name: venv.dev.python.foo

  uv-python:
    external: true
    name: uv-python
```

I valori sono illustrativi: il renderer li produce dal contesto e dai due file runtime.

Non definiamo:

```
command: ["sleep", "infinity"]
```

e non avviamo `dev` con `up -d`. Lo creiamo esclusivamente quando serve la sessione, tramite `run`.

3. Comando equivalente al run attuale

Lo schema di esecuzione diventa:

```
compose_args=(
  --project-name "$COMPOSE_PROJECT_NAME"
  --project-directory "$PROJECT_DIR"
  -f "$COMPOSE_FILE"
)

docker compose "${compose_args[@]}" \
```

```
  run \
  --rm \
  --name "$CONTAINER_NAME" \
  --service-ports \
  dev \
  "${CONTAINER_CMD[@]}"
```

Qui:

- `--name` mantiene il nome del container scelto da `docker-dev`;
- `--rm` mantiene il comportamento effimero;
- `--service-ports` applica le pubblicazioni dichiarate nel servizio;
- il comando dopo `dev` sostituisce il comando predefinito, senza richiedere un override dell'entrypoint.[1]

Se `CONTAINER_CMD` è vuoto, non aggiungiamo `bash`: lasciamo che l'entrypoint dell'immagine scelga il proprio comportamento senza argomenti.

Per l'esecuzione batch, va definita una modalità senza TTY, usando `-T`; Compose gestisce l'allocazione del terminale anche in base al contesto di esecuzione.[1]

Nome fisso e concorrenza

Preservare `CONTAINER_NAME` significa anche mantenere il vincolo di una sola sessione con quel nome. In presenza di un container omonimo proporrei un errore esplicito, anziché rimuoverlo forzatamente.

Questa sarebbe una modifica intenzionale rispetto alla vecchia politica di rimozione preventiva, non un limite di Compose.

4. Caso con servizi aggiuntivi

La differenza fra progetto singolo e progetto con servizi diventa soltanto la preparazione delle dipendenze:

```
Solo sviluppo:
    run --rm dev

Sviluppo + servizi:
    up dei soli servizi selezionati
    attesa readiness
    run --rm dev
```

Per esempio:

```
docker compose "${compose_args[@]}" up -d db cache

# Dopo aver verificato la readiness:
docker compose "${compose_args[@]}" \
  run --rm --no-deps \
```

```
    --name "$CONTAINER_NAME" \
    --service-ports \
    dev "${CONTAINER_CMD[@]}"
```

`--no-deps` è appropriato quando l'orchestratore ha già preparato e verificato le dipendenze; impedisce al comando `run` di avviarle nuovamente come parte della propria gestione. [1]

Senza servizi selezionati, il primo comando viene saltato. Non bisogna eseguire `up -d` senza nomi, perché avvierebbe anche il servizio di sviluppo.

5. Che cosa cambia nell'architettura

Eliminando il backend legacy, possiamo togliere:

- il traduttore del modello runtime in opzioni `docker run`;
- il dispatcher legacy/Compose;
- la doppia verifica di equivalenza fra due rappresentazioni runtime;
- la compatibilità permanente con `docker-run-extra-args`.

Rimangono:

```
runtime di linguaggio
          +
runtime di progetto
          ↓
risoluzione e validazione
          ↓
configurazione Compose di dev
          +
configurazione dei servizi selezionati
          ↓
docker compose
```

La pipeline approvata resta utilizzabile:

```
10.setup-volumes.sh
20.build.sh
30.services.sh
40.run.sh
```

La preparazione comune del runtime avviene prima della pipeline. `30.services.sh` aggiunge e prepara gli eventuali servizi; `40.run.sh` usa sempre Compose.

Questo non impone di eliminare tutti i comandi Docker diretti: `10.setup-volumes.sh` e `20.build.sh` possono continuare a usare `docker volume`, un container temporaneo di inizializzazione e `docker build`. "Solo Compose" significa, in questa fase, un unico backend di esecuzione del progetto, non una riscrittura obbligatoria di ogni operazione di preparazione.

6. La differenza più importante da verificare: la rete

L'equivalenza applicativa non implica identità della configurazione di rete. La rete dello stack Compose va considerata parte della nuova architettura: permette al container di sviluppo di raggiungere i servizi tramite i loro nomi.

Perciò il collaudo deve verificare:

- accesso ai servizi dello stack;
- accesso all'host, se richiesto;
- DNS e connettività esterna;
- pubblicazione delle porte dichiarate;
- eventuali esigenze di indirizzi o alias di rete.

Non è necessario forzare la vecchia rete soltanto per ottenere un'identità formale: occorre preservare il comportamento richiesto dal progetto.

7. Conseguenza sulla scelta di `runtime.yaml`

Con un unico backend, non serve più progettare uno schema portabile verso `docker run`. Possiamo avvicinare i campi supportati alla terminologia Compose e lasciare a Compose la validazione finale, mantenendo nel nostro resolver soltanto:

- i due livelli linguaggio/progetto;
- i riferimenti controllati al contesto;
- la risoluzione dei percorsi;
- le regole semplici di composizione;
- i controlli sui mount standard e sui conflitti.

Manteniamo quindi la semplificazione appena approvata: niente lifecycle o hook nel runtime, mount essenziali e preparazione delle risorse negli script.

La scelta "solo Compose" è dunque concretamente praticabile e riduce la complessità. Il suo criterio di accettazione sarà la stessa esperienza di esecuzione — immagine, entrypoint, utente, mount, comando, interattività e persistenza — con le differenze intenzionali di rete e gestione dello stack documentate e collaudate.

⁂

1. https://docs.docker.com/reference/cli/docker/compose/run/
2. https://docs.docker.com/reference/dockerfile/
3. https://docs.docker.com/reference/cli/docker/compose/up/
4. https://docs.docker.com/reference/cli/docker/compose/ps/
5. https://docs.docker.com/reference/cli/docker/compose/build/
6. https://docs.docker.com/compose/gettingstarted/
7. https://docs.docker.com/reference/compose-file/volumes/

8. https://docs.docker.com/reference/cli/docker/compose/down/
9. https://docs.docker.com/compose/support-and-feedback/faq/
10. https://docs.docker.com/get-started/docker-concepts/running-containers/overriding-container-defaults/
11. https://docs.docker.com/reference/cli/docker/service/create/
12. https://docs.docker.com/reference/cli/docker/compose/pull/
13. https://docs.docker.com/reference/cli/docker/compose/restart/
14. https://docs.docker.com/reference/compose-file/services/
15. https://docs.docker.com/reference/compose-file/